UniCEUB - CENTRO UNIVERSITÁRIO DE BRASÍLIA

Faculdade de Tecnologia e Ciências Sociais Aplicadas

CURSO: Ciência da Computação

DISCIPLINA: Arquitetura de Sistemas Distribuídos

PROFESSOR: Msc. Fabiano Mariath D´Oliveira

Previsão de Estabilidade em Sistemas Distribuídos: Um Modelo de Inteligência Artificial para Controlar Recursos e Picos de Carga Causadas por Fatores Externos

ALUNOS

22105593 - Lucas Zoser Nunes Costa

22101570 - Leonardo Areias Rodovalho

22103833- Leonardo Benttes Almeida Placido dos Santos

22201914 - Pedro Henrique Moreira da Silva

Brasília, 10 de Abril de 2024

1. Introdução

A partir da década de 2010, a dependência da sociedade em serviços e sistemas online intensificou-se exponencialmente, tendo relação direta com dois fatores: a proliferação de aplicativos móveis e a “Internet das Coisas” (IoT) **[1]**. Com isso, o acesso tornou-se muito mais fácil e rápido, ou seja, agora é possível acessar a internet de qualquer lugar, qualquer dispositivo, a qualquer hora. Isso trouxe novas perspectivas para o mundo do trabalho, lazer, saúde e segurança. Sendo assim, agora as empresas pensam em novas soluções para seus processos e produtos digitais, tornando-os mais acessíveis e mais lucrativos **[2]**.

Vive-se hoje em uma sociedade digital, então é natural a necessidade de automatizar e digitalizar as informações e nossas atividades cotidianas. Essa transformação causou também a dependência dessas tecnologias. Antigamente, era comum a prática de pegar uma fila em uma agência bancária para pagar um boleto. Hoje em dia, faz-se isso em poucos segundos com um aplicativo bancário. Assim também, antigamente era impossível estudar a distância, trabalhar remotamente ou poder assistir praticamente qualquer coisa em casa. Logo, a presença desses aplicativos, dispositivos e serviços online tornou-se crucial na vida das pessoas, permitindo que diversos processos "antigos" sejam implementados de forma distinta, através de softwares que facilitam o acesso à informação e a realização de tarefas diversas. Essa mudança traz consigo diversos benefícios, como praticidade, eficiência e acessibilidade. Mas também gera dependência dessas tecnologias e pede que elas funcionem incessantemente, ou seja, precisam sempre estar disponíveis para seus usuários, além disso, devem desempenhar bem suas funções e ser escaláveis para a quantidade de usuários, seguindo os conceitos de sistemas distribuídos **[3]**.

1.1. Problema de Pesquisa

O cerne do problema investigado nesta pesquisa está nessa crescente dependência das pessoas em relação aos serviços e aplicações online, os quais se tornaram essenciais em suas vidas diárias. Contudo, essa dependência está sujeita a uma vulnerabilidade preocupante: a volatilidade desses serviços. À medida que cada vez mais aspectos de nossas vidas, desde transações financeiras até comunicação e entretenimento, são

transferidos para o ambiente digital, a lentidão desses serviços tem impactos significativos. Quando essas aplicações não estão disponíveis, seja devido a falhas técnicas, ataques cibernéticos ou picos de carga não previstos, as pessoas enfrentam inconvenientes consideráveis e, em alguns casos, prejuízos financeiros. Um exemplo disso foi o Ataque de ransomware à Colonial Pipeline, em que um grande fornecedor de petróleo e gás para a costa leste dos EUA foi derrubado através desse ataque, o que provocou uma instabilidade no mercado de combustíveis e contribuiu para o aumento da inflação **[4]**.

O desafio, portanto, está em garantir a estabilidade de forma contínua e confiável desses serviços, mesmo diante das pressões e demandas cada vez maiores. Essa questão é ainda mais complexa devido à diversidade de serviços e aplicações distribuídas, cada uma com seus próprios requisitos de desempenho e disponibilidade. Assim, a necessidade de abordar esse problema torna-se crucial não apenas para garantir a conveniência e eficiência das operações online, mas também para preservar a confiança e a segurança dos usuários em um mundo cada vez mais digitalizado.

1.2. Alternativas de Solução

Existem diversas abordagens para garantir que as aplicações permaneçam operacionais e acessíveis, mesmo diante de momentos de alta demanda de carga, especialmente em ambientes distribuídos. Alguns estudos específicos investigaram essas abordagens detalhadamente. Por exemplo, o estudo "A Comparative Study of Grid Load Balancing"**[5]** analisou diferentes técnicas de balanceamento de carga em sistemas distribuídos, como Round Robin, Weighted Round Robin e Least Connections. Os pesquisadores descobriram que, enquanto o Round Robin é eficaz para distribuir recursos, o Weighted Round Robin permite uma alocação mais inteligente de recursos. A escolha da estratégia depende das necessidades específicas do sistema, destacando a importância de selecionar cuidadosamente o algoritmo de balanceamento de carga para otimizar o desempenho e a disponibilidade em sistemas distribuídos.

Outra abordagem importante foi investigada no estudo "Microservices: A Systematic Mapping Study"**[6]**, que analisou a literatura sobre arquiteturas de microsserviços. Os pesquisadores concluíram que os microsserviços oferecem flexibilidade, escalabilidade e independência de implantação. No entanto, a comunicação entre microsserviços e a gestão

de dados distribuídos apresentam desafios significativos. Assim, a adoção de microsserviços deve ser cuidadosamente planejada, considerando os trade-offs entre benefícios e desafios.

Estas abordagens destacam a importância da previsão de estabilidade em sistemas distribuídos e a utilização de modelos de inteligência artificial para controlar recursos e picos de carga causados por fatores externos.

Além disso, o estudo "Optimized Service Level Agreement Establishment in Cloud Computing" **[7]** concentrou-se na otimização do estabelecimento de Acordos de Nível de Serviço (SLAs) em computação em nuvem. Os pesquisadores descobriram que a previsão de demanda e a escalabilidade automática são essenciais para atender aos SLAs. Algoritmos de machine learning e análise de dados históricos permitem uma alocação eficiente de recursos. Conclui-se que a otimização dos SLAs é fundamental para garantir a qualidade do serviço em ambientes de nuvem.

Essas pesquisas forneceram insights valiosos sobre as estratégias mais eficazes para lidar com o desafio da disponibilidade das aplicações e serviços, o que leva em conta alguns fatores, tal como o gerenciamento de carga. Destacando a importância de abordagens como balanceamento de carga, arquiteturas de microsserviços e otimização de SLAs para garantir a disponibilidade contínua e a performance otimizada dos sistemas distribuídos.

1.3. Método de Pesquisa

O método de pesquisa deste trabalho consistirá nas seguintes etapas **[8]**:

- Revisão da Literatura: Será realizada uma revisão sistemática e crítica da literatura relevante na área de previsão de picos de carga e sistemas de autoajuste. Essa revisão da literatura será fundamental para identificar lacunas nas soluções existentes, definir o escopo da pesquisa e compreender o estado da arte, seguindo as diretrizes estabelecidas por especialistas na área.

- Modelo Preditivo Baseado em Inteligência Artificial: Será desenvolvido um modelo preditivo utilizando técnicas de Inteligência Artificial, como aprendizado de máquina, para prever picos de carga com base em dados históricos. O objetivo é explorar algoritmos, seleção de recursos e validação cruzada, seguindo as melhores práticas estabelecidas por especialistas na construção e avaliação de modelos. Este modelo visa otimizar a alocação de recursos e melhorar a resiliência dos sistemas distribuídos em resposta a fatores externos.

- Simulação e Teste em Ambiente Controlado: A validação experimental será realizada por meio de simulações em ambiente controlado. Serão criados cenários realistas de estresse em aplicações web para avaliar o desempenho do sistema de ajuste de recursos, levando em consideração fatores como disponibilidade e estado atual da aplicação.

- Alternativas para Validação: Caso o teste em protótipo em escala real não seja viável devido a restrições de tempo ou recursos, serão consideradas alternativas, tais como testes em ambientes virtuais para simular cenários reais com menor custo e risco, geração de dados sintéticos para testar o sistema e a técnica de validação cruzada, dividindo os dados históricos em conjuntos de treinamento e teste para avaliar o desempenho do modelo, de acordo com orientações estabelecidas por especialistas na área.

1.4. Objetivo do Trabalho

O objetivo deste trabalho é alertar quando aplicações web estão sendo sobrecarregadas. Com isso, será indicado se essa aplicação está em estado “saudável” ou “não saudável”, o que significa que ela deve ser escalonada verticalmente, com alocamento de recursos. Para isso, serão desenvolvidos modelos de inteligência artificial (IA) para prever e identificar picos de carga e estados de aplicações em sistemas distribuídos. O sistema será desenvolvido em 2 meses, seguido de uma fase piloto de 1 mês para avaliar sua efetividade.

Específico: Desenvolver um modelo de aprendizado de máquina para prever e gerenciar picos de carga em sistemas distribuídos.

Mensurável: Emitir um alerta com uma métrica que indica o estado atual de uma aplicação para ser tomada uma ação efetiva de acordo com a realocação de recursos.

Atingível: Criar um plano de implementação detalhado, com prazos e recursos definidos, utilizando tecnologias de IA e colaborando com especialistas na área.

Relevante: Melhorar a estabilidade e eficiência das plataformas online, proporcionando uma solução que pode futuramente ser desenvolvida para melhorar a experiência do usuário, gerar mais confiabilidade e reduzir custos de operação para empresas.

Temporal: Concluir o desenvolvimento e a fase de testes do sistema em 2 meses, seguido de uma fase piloto de 1 semana para avaliar a efetividade da solução em um ambiente real.

2 - Referencial Teórico

2.1 - Previsão de Carga de Rede e Carga de Aplicação

A previsão de carga de rede é o processo de estimar a quantidade de dados que será transmitida através de uma rede em um determinado período de tempo. Isso é essencial para garantir que os sistemas distribuídos, como os serviços online, possam se adaptar às flutuações na demanda e evitar interrupções no serviço. Ao entender e antecipar os padrões de carga, medidas proativas podem ser adotadas para evitar congestionamentos, minimizar o tempo de inatividade e garantir uma experiência do usuário consistente.

De acordo com Oliveira, Barbar e Soares (2015) **[9]**, em seu artigo sobre previsão de carga de rede, o carga de rede é uma série temporal, ou seja, ele é uma sequência de dados medidos regularmente em intervalos de tempo uniformes. Para o carga de rede, esses dados sequenciais são os bits transmitidos em algum dispositivo de rede em um certo período de tempo. Uma série temporal pode ser um processo estocástico ou um processo determinístico. Para prever uma série temporal são necessários modelos matemáticos que realmente representam as características estatísticas do carga amostrado. A escolha do método de previsão para aplicações adaptativas que necessitam processamento em tempo-real deve levar em consideração o horizonte de predição, custo computacional, erro de predição e o tempo de resposta.

Realizar uma previsão de carga ou carga, em uma aplicação é fundamental para o desenvolvimento deste trabalho, pois assim, busca-se entender quando uma aplicação está em apuros, prestes a ficar inativa e indisponível. Ao prever com precisão esses picos, pode-se dimensionar os recursos de forma adequada, com alocação mais capacidade quando necessário e economizando recursos quando a demanda diminui, uma prática já conhecida em sistemas de computação em nuvem, denominada escalonamento automático de recursos **[10]**. Isso não apenas melhora a experiência do usuário, reduzindo as interrupções, mas também aumenta a eficiência operacional e a receita das empresas, evitando perdas causadas por tempo de inatividade não planejado.

A previsão refere-se à capacidade de estimar ou antecipar eventos futuros com base em dados históricos e padrões identificados. A inteligência artificial desempenha um papel crucial na previsão de carga de rede, pois os algoritmos de aprendizado de máquina podem analisar grandes volumes de dados, identificar padrões complexos e fazer previsões precisas sobre o comportamento futuro do carga. Isso permite uma adaptação dinâmica dos recursos de rede para atender às demandas variáveis dos usuários, melhorando assim a eficiência e a confiabilidade dos sistemas distribuídos **[9]**.

Para este trabalho, serão exploradas algumas técnicas para a previsão de carga de aplicação. Após revisar a literatura e analisar pesquisas nesta área, descobrimos a técnica de previsão LSTM (Long Short-Term Memory), uma arquitetura de rede neural recorrente projetada para lidar com sequências temporais. As LSTMs são particularmente eficazes para lidar com dados relacionados à previsão de carga, pois têm a capacidade de reter informações contextuais ao longo da série temporal. Ao contrário das redes neurais recorrentes tradicionais, que enfrentam dificuldades para capturar dependências de longo prazo devido ao problema do gradiente desvanecente, as LSTMs superam essa limitação incorporando mecanismos de memória que permitem a retenção de informações por períodos prolongados **[11][12]**.

A utilização de LSTMs nos estudos revisados é justificada pela sua capacidade de aprender e reconhecer padrões complexos na carga da rede, a qual é intrinsecamente dinâmica e apresenta variações significativas ao longo do tempo. Treinar um modelo utilizando uma série temporal dos dados históricos de carga da rede permite ao sistema identificar tendências e comportamentos sazonais **[13]**. Além disso, a arquitetura LSTM pode ser configurada com múltiplas camadas e neurônios para otimizar a precisão das previsões.

Além das LSTMs, foram investigados o modelo de árvores de decisão. Este modelo, um algoritmo de aprendizado de máquina supervisionado, é utilizado tanto para classificação quanto para regressão, e mostra-se especialmente útil para prever categorias discretas, como por exemplo: “Devo alocar mais recursos para esta aplicação?” (Sim ou Não). Após analisar a implementação desses modelos, foi observado que geralmente eles são divididos em duas fases principais: treinamento e previsão. Durante o treinamento, as

amostras de dados são codificadas e integradas pelo algoritmo de extração de características. Em seguida, essas características são utilizadas pelo algoritmo de árvore de decisão para treinar o modelo preditivo. Durante a fase de previsão, dadas as amostras de carga, segue-se um processo similar de codificação das amostras, e um modelo treinado é empregado para prever a saúde da aplicação. O modelo de árvore de decisão atribui uma pontuação a cada amostra de carga, indicando a probabilidade de sua saúde estar em risco ou se ela necessita de mais recursos. Por exemplo, uma pontuação abaixo de 2 indica que a aplicação está em risco, e abaixo de 1 sugere que ela precisa de alocação adicional de recursos.

Assim, nos trabalhos pesquisados, notou-se um padrão relacionado a construção dos modelos para conseguir realizar uma previsão :

Coleta de Dados: Captura de dados históricos de uso de recursos em intervalos regulares.

Pré-processamento de Dados: Limpeza e normalização dos dados para remover ruídos e padronizar as entradas **[11]**.

Divisão dos Dados: Separação dos dados em conjuntos de treinamento, validação e teste.

Treinamento do Modelo: Utilização do conjunto de treinamento para ajustar os pesos da rede LSTM.

Validação e Teste: Avaliação do modelo usando os conjuntos de validação e teste para ajustar hiperparâmetros e verificar a precisão da previsão **[14][15]**.

Implementação em Tempo Real: Integração do modelo treinado no sistema de monitoramento de rede para previsões em tempo real.

Esse processo facilitará a codificação futura e será a primeira tentativa de construção de um modelo de previsão de carga, capaz de ajustar dinamicamente os recursos da rede conforme as previsões **[16][17]**. Além disso, ao prever com precisão quando uma aplicação está em apuros, pode-se dimensionar os recursos de forma adequada, com alocação de mais capacidade quando necessário e economia de recursos quando a demanda diminui, uma prática já conhecida em sistemas de computação em nuvem, denominada escalonamento automático de recursos **[18]**.

2.2 Sistemas de Autoajuste (Auto-scaling)

O autoajuste, ou escalabilidade automática, refere-se à capacidade de um sistema de aumentar ou diminuir automaticamente os recursos computacionais em resposta à demanda de carga. Isso é essencial em ambientes de computação em nuvem, onde a demanda por recursos pode variar significativamente ao longo do tempo. O objetivo principal do autoajuste é garantir que os aplicativos tenham os recursos necessários para manter um desempenho ideal, ao mesmo tempo em que minimizam os custos ao não provisionar recursos desnecessários **[14][16]**. Isso é realizado sem intervenção manual, garantindo que o sistema esteja sempre otimizado para as condições atuais.

A escalabilidade automática é uma implementação específica de autoajuste em que a infraestrutura de TI ajusta a capacidade de recursos (como CPU, memória e largura de banda) para corresponder à demanda do aplicativo ou serviço em execução **[17][18]**.

As políticas e algoritmos de escalonamento são essenciais para a gestão eficiente de recursos em sistemas computacionais, especialmente em modelos de IA que preveem o carga de rede. Elas ajustam dinamicamente a alocação de recursos com base em métricas operacionais, garantindo resposta adequada a variações de carga. As abordagens comuns incluem escalonamento baseado em limite, passo a passo e preditivo, cada uma com vantagens específicas para diferentes contextos **[19]**. Algumas dessas políticas e algoritmos pesquisados para auxiliar esse trabalho foram:

- Política de Escalonamento Baseada em Limite (Threshold-based Scaling): Aumenta ou diminui os recursos com base em métricas predefinidas, como a utilização da CPU ou a taxa de transferência de rede. Por exemplo, se a utilização da CPU exceder 70% por um determinado período, mais instâncias serão adicionadas **[19]**.
- Política de Escalonamento Passo a Passo (Step Scaling): Ajusta os recursos em etapas com base na magnitude da carga. Isso significa que maiores aumentos na carga resultarão em adições maiores de recursos, enquanto cargas menores resultam em ajustes menores **[20]**.
- Escalonamento com Base em Previsão (Predictive Scaling): Utiliza algoritmos de aprendizado de máquina para prever futuras cargas de trabalho com base em dados históricos, ajustando os recursos antes que a demanda aumente **[20]**.

Um exemplo analisado de auto-scaling é a Amazon Web Services (AWS) Auto Scaling, que permite aos usuários configurar políticas de escalabilidade para suas aplicações. O AWS Auto Scaling monitora os aplicativos e ajusta automaticamente a capacidade para manter um desempenho estável e previsível ao menor custo possível **[17]**. Outro exemplo é a implementação no Google Cloud Platform (GCP), que oferece escalabilidade automática para seus serviços de computação, ajustando automaticamente o número de instâncias de máquina virtual com base na demanda do usuário **[18]**.

Nosso primeiro modelo será relacionado com a noção de escalabilidade de uma aplicação, ou seja, ele entenderá se uma aplicação precisa passar pelo processo de auto-scaling. Assim, ao prever os picos e vales no carga de rede, nosso modelo permitirá ajustes proativos na infraestrutura, garantindo que os recursos estejam sempre alinhados com a demanda esperada. Isso não apenas reduz o risco de sobrecarga do sistema durante períodos de alta demanda, mas também economiza custos ao evitar o provisionamento excessivo de recursos em períodos de baixa utilização **[16]**. Não vamos implementar o auto-scaling diretamente em nenhuma solução de cloud computing, contudo deixaremos uma função no código que pode acionar essa medida, podendo ser implementada posteriormente, já que nosso foco é fazer uma previsão.

2.3 Algoritmos de Aprendizado de Máquina

Seguindo o princípio de algoritmo, que são sequências de regras ou processos, algoritmos de aprendizado de máquina são essas instruções aplicadas em um sistema de IA para a realização de tarefas com o objetivo de descobrir novas informações com base em dados de entrada, produzindo uma saída com valor **[21]**. Para a produção desses valores de saída é necessária uma ferramenta que ache padrões nesses dados e entenda o que estes significam, essas ferramentas são os algoritmos de aprendizado de máquina, cada um serve seu propósito se adaptando principalmente aos dados, às regras de negócio e ao que se deseja obter.

Em um contexto de previsão de disponibilidade em sistemas distribuídos, que é o que buscamos, um algoritmo se destaca, o de redes neurais LSTMs. Ele realiza um aprendizado constante e aprende como um humano aprenderia, além de se destacar devido

à suas camadas escondidas que diferem de outros algoritmos de predição como o de regressão linear, que utiliza apenas de nós de entrada e saída para realizar predições **[22]**.

Redes Neurais são ótimas ferramentas para lidar com séries temporais, dessa forma são implementadas recorrentemente em problemas de previsão de carga, que possuem uma constância de séries temporais. Dentre as diversas redes neurais a melhor e mais utilizada neste tópico são as RNNs que introduzem uma estrutura recorrente à rede neural, possibilitando que os neurônios lembrem da informação que foi tratada anteriormente e assim influencie na saída do momento atual do neurônio **[23]**. Mais especificamente, as LSTMs são um tipo específico de RNNs que são compostas de unidades chamadas de blocos de memórias. Um bloco de memória contém células de memória com conexões também para elas mesmas (recorrente) possibilitando o armazenamento do estado temporal daquela rede **[24]**.

As LSTMs são muito apropriadas para lidar com cargas de rede, estudos mostram que com o pré-processamento de dados e técnicas de alimentação de RNNs, atinge-se altos níveis de predição e de acurácia com apenas alguns segundos de computação **[24]**. Além disso, essa técnica mostra-se melhor performática em relação a algoritmos de predição tradicionais como métodos lineares e até mesmo outros tipos de redes neurais como as de feed forward (em que o processamento é somente em uma direção, do nó de entrada para o nó de saída), superando-as em diversos níveis de magnitude **[24]** **[25]**.

A preparação de dados é uma etapa necessária em qualquer projeto de Machine Learning, com impacto direto na precisão das previsões. Ferramentas como Pandas e NumPy são amplamente utilizadas para a manipulação de dados, permitindo operações complexas de limpeza e transformação de dados de maneira eficiente **[28]**.

A geração de dados sintéticos é uma técnica útil quando dados reais são insuficientes ou indisponíveis. Técnicas como oversampling podem equilibrar classes desbalanceadas, melhorando a robustez dos modelos. Iremos seguir essas melhores práticas e utilizar as ferramentas mencionadas acima no referencial teórico para garantir que os dados estejam em um formato adequado para o treinamento de modelos de previsão. A normalização e padronização dos dados serão aplicadas para assegurar que diferentes características tenham uma escala uniforme.

2.4 Avaliação de Desempenho do Sistema

A avaliação de desempenho do sistema é um tópico amplamente discutido na literatura, com diversos estudos abordando as melhores práticas e ferramentas para garantir eficiência e confiabilidade. De acordo com Oliveira, Barbar e Soares (2015) **[9]**, as métricas mais utilizadas incluem latência, throughput e disponibilidade, que são essenciais para avaliar a performance de sistemas de rede.

Ferramentas como Prometheus e Grafana são comumente utilizadas em outros projetos para monitoramento em tempo real. Prometheus permite a coleta de métricas detalhadas, como a utilização de CPU e memória, enquanto Grafana facilita a visualização desses dados através de dashboards interativos **[26]** **[27]**.

Estudos de caso como "Understanding performance interference in multi-tenant cloud databases and web applications" **[26]** e "Load Testing Websites" **[27]** demonstram a aplicação prática dessas ferramentas em ambientes reais, destacando os desafios e soluções encontrados na prática.

3 - Desenvolvimento

Este artigo propõe o desenvolvimento de um sistema de inteligência artificial (IA) para prever e gerenciar o estado de uma aplicação em sistemas distribuídos, com o objetivo de alertar quando uma aplicação está em uma situação de estresse, gerando aumentando a disponibilidade para 95%, e, consequentemente, melhorando a experiência do usuário e a receita das empresas.

Inicialmente, a fase de planejamento estabelecerá os requisitos específicos do projeto, identificando as métricas de sucesso e os critérios de aceitação. A formação da equipe multidisciplinar, composta por especialistas em IA, desenvolvimento de software e operações de TI, será crucial para o sucesso do projeto.

A fase de desenvolvimento começa com a coleta e análise de dados históricos de carga dos sistemas distribuídos, essenciais para treinar e validar o modelo de ML. A seleção de algoritmos apropriados e a otimização do modelo garantirão previsões precisas de picos de carga. Paralelamente, será desenvolvido um sistema automatizado de gerenciamento de recursos, capaz de ajustar a capacidade em tempo real com base nas previsões do modelo. A integração do sistema com plataformas de computação em nuvem permitirá uma resposta ágil e eficiente aos picos de carga.

Após a conclusão do desenvolvimento, a fase de teste e validação será conduzida em um ambiente controlado, simulando picos de carga para avaliar o desempenho do sistema. Ajustes finais serão feitos com base nos resultados dos testes. A fase piloto implementará o sistema em um ambiente de produção controlado, permitindo a coleta de feedback dos usuários e a avaliação da redução do tempo de inatividade e da melhoria da experiência do usuário.

Desta forma, o nosso modelo de Machine Learning deve entender quando uma aplicação está passando por problemas através de previsões constantes do estado da mesma. Para isso, devemos sempre responder duas perguntas críticas relacionadas à gestão de recursos em aplicações web:

1. *Qual o estado atual da aplicação?*
2. *Essa aplicação vai sofrer um pico de carga?*

Para essa abordagem, foram escolhidos dois modelos: uma Árvore de Decisão para prever o estado atual da aplicação **[29]**, e uma Rede Neural Recorrente(RNN) LSTM para prever picos de carga **[24]**. A escolha desses modelos baseou-se em estudos anteriores mencionados na revisão de literatura e na escolha de uma base de dados utilizados para treinamento e teste desses modelos.

3.1 Escolha dos Modelos

A Árvore de Decisão foi escolhida pela sua simplicidade e eficácia em problemas de classificação, visto que vai ser tomada uma decisão: alocação ou não alocação de recursos

para uma aplicação web. A bibliografia suporta seu uso em diversas aplicações, conforme descrito na documentação do scikit-learn **[30]**.

Assim, após testarmos o modelo de Árvore de Decisão, a depender dos resultados, pensamos em utilizar o modelo de Random Forest. Tal técnica pode ser mais interessante, pelo fato de utilizar múltiplas árvores de decisão, trazendo mais robustez e possivelmente melhorando a previsibilidade do modelo. Cada árvore no ensemble é construída a partir de uma amostra aleatória da base de dados original e, durante a divisão dos nós, apenas um subconjunto aleatório de características é considerado. Isso ajuda a reduzir a variância do modelo e mitigar problemas de overfitting comuns em modelos individuais de árvores de decisão. Estudos demonstram que Random Forest é eficaz em várias aplicações, incluindo classificação, regressão e detecção de anomalias, devido à sua capacidade de capturar interações complexas entre características e generalizar melhor em dados não vistos **[31]**.

Para prever picos de carga, trataremos primeiramente o modelo de RNN simples com LSTM **[23]**, devido à sua capacidade de modelar sequências temporais e capturar dependências de longo prazo nos dados. Estudos demonstram a eficácia das LSTMs em tarefas de previsão de séries temporais e detecção de anomalias em fluxos de dados de rede **[24]**. Contudo, apesar da grande utilidade desse tipo de modelo, optamos por considerar outras técnicas também. Modelos como ARIMA **[32]**, Prophet **[33]**, Perceptron **[34]** ou até mesmo abordagens híbridas, podem oferecer benefícios específicos dependendo da natureza dos dados e da tarefa. Por exemplo, modelos de regressão linear podem ser mais interpretáveis, enquanto modelos baseados em aprendizado de máquina podem ser mais adequados para capturar padrões complexos e não lineares nos dados. Então, a depender do desempenho das LSTMs, consideraremos outras abordagens.

Essas serão as duas primeiras técnicas de previsão que utilizaremos em nosso desenvolvimento, testaremos os resultados e veremos se será adequado para nossa solução. Assim, essas bibliotecas serão utilizadas no desenvolvimento e implementação do código:

- Google Colab: Será utilizado para a construção e treinamento inicial do modelo de IA. O Google Colab oferece um ambiente de notebooks Jupyter baseado na nuvem,

que suporta GPUs gratuitas e facilita a colaboração e o compartilhamento de código e resultados.

- TensorFlow: Esta biblioteca open-source será empregada para a construção das RNNs e LSTM. O TensorFlow é amplamente utilizado para tarefas de aprendizado profundo devido à sua flexibilidade e eficiência.
- Keras: Uma API de alto nível para redes neurais, que roda sobre o TensorFlow. O Keras simplifica a criação e experimentação de modelos de aprendizado profundo.
- Pandas e NumPy: Para manipulação e análise de dados. Essas bibliotecas são essenciais para o pré-processamento e limpeza dos dados históricos de carga de rede.
- Matplotlib e Seaborn: Ferramentas de visualização de dados que serão usadas para explorar e entender os padrões de carga de rede durante o processo de desenvolvimento do modelo.
- Scikit-Learn: Biblioteca para implementação do algoritmo de árvore de decisão. A scikit-learn é uma biblioteca de aprendizado de máquina de código aberto para a linguagem de programação.

3.2 Desenvolvimento de Modelo: Árvore de Decisão

O desenvolvimento baseado no modelo de árvore de decisão, utiliza da base de dados (dataset) encontrada no site Kaggle **[35]**, que diz respeito a uma amostra do rastreamento das cargas de trabalho em execução em oito clusters de computação do Google Borg (Gerenciador de Clusters) no mês de maio de 2019. Nessa base, conseguimos informações detalhadas sobre cada envio de trabalho, decisão de agendamento e uso de recursos, podendo ser extraída uma análise valiosa a respeito das operações que são realizadas em um ambiente de computação em nuvem real e de grande escala e aplicando essa compreensão no entendimento do estado de uma aplicação e em um pico de carga iminente.

Antes de utilizar o dataset para treinar o modelo, uma análise exploratória dos dados foi conduzida para entender melhor suas características. O fator mais importante em toda essa análise foi a identificação de eventos e instâncias, que possuem um ciclo de vida próprio, ou seja, cada instância tem diferentes estados e transições de eventos ao decorrer do tempo. Sendo assim, identificamos nesses eventos o comportamento de um aplicativo,

indicando rótulos para saber sua performance ao longo do tempo. Desta maneira, o dataset também inclui informações detalhadas sobre o uso de recursos, como CPU e memória, coletados em intervalos regulares de 5 minutos, fornecendo dados sobre:

- **average_usage_memory**: Uso médio de memória durante a execução do trabalho.
- **maximum_usage_memory**: Uso máximo de memória alcançado pelo trabalho.
- **assigned_memory**: Memória atribuída a cada trabalho.
- **cpu_usage**: Uso da CPU, medido em unidades de CPU normalizadas (NCUs).

Dentre os eventos citados, os mais importantes que ocorrem durante o ciclo de vida de uma instância são:

- **SUBMIT**: A instância é submetida para execução.
- **QUEUE**: A instância está na fila de espera para ser agendada.
- **SCHEDULE**: A instância é agendada para execução em uma máquina.
- **UPDATE_PENDING**: Atualizações de requisitos de recursos ou prioridade enquanto a instância está na fila.
- **UPDATE_RUNNING**: Atualizações de requisitos de recursos ou prioridade enquanto a instância está em execução.
- **EVICT**: A instância é desalojada devido a várias razões, como prioridade de outros trabalhos ou falha na máquina.
- **FINISH**: A instância é finalizada com sucesso.
- **KILL**: A instância é cancelada pelo usuário ou por um programa supervisor.
- **LOST**: A instância foi perdida, geralmente devido a uma falha de registro de eventos.

Assim, o dataset possui 34 colunas, das quais seis foram selecionadas como relevantes para construção do nosso algoritmo, em baixo, estão informações referentes ao significado de cada uma dessas colunas e por que elas são relevantes para a construção do nosso modelo:

1. scheduling_class (Classe de Agendamento):

- Esta coluna indica a classe de agendamento da tarefa, que representa quão sensível à latência é a execução da tarefa. As classes variam de 0 a 3, onde 3

representa tarefas mais sensíveis à latência (por exemplo, servindo solicitações de usuários em tempo real) e 0 representa tarefas menos críticas (por exemplo, análises não críticas ou desenvolvimento).

- A classe de agendamento é crucial porque tarefas com diferentes sensibilidades à latência têm diferentes requisitos de recursos e comportamento de execução. Isso ajuda o modelo a entender quais tarefas precisam de mais recursos imediatamente e quais podem esperar.

2. priority (Prioridade de Execução):

- Esta coluna representa a prioridade da tarefa. Tarefas com maior prioridade (valores maiores) têm preferência na alocação de recursos em relação a tarefas com menor prioridade.
- A prioridade é um indicador direto de quais tarefas devem receber recursos primeiro. Isso é essencial para prever como os recursos serão distribuídos e gerenciados no cluster.

3. assigned_memory (Memória Atribuída):

- Esta coluna mostra a quantidade de memória que foi atribuída à tarefa no momento de sua execução.
- A memória atribuída é uma medida dos recursos disponíveis para a tarefa. É importante para o modelo prever a necessidade de memória e garantir que as tarefas tenham recursos suficientes para serem executadas eficientemente.

4. average_usage_memory (Uso Médio de Memória):

- Esta coluna indica a quantidade média de memória utilizada pela tarefa durante sua execução.
- O uso médio de memória ajuda a identificar o comportamento típico da tarefa em termos de consumo de memória, o que é útil para ajustar as alocações de recursos de forma eficiente e evitar desperdícios.

5. maximum_usage_memory (Uso Máximo de Memória):

- Esta coluna mostra o uso máximo de memória alcançado pela tarefa durante sua execução.

- O uso máximo de memória é crítico para garantir que as alocações de memória não sejam subestimadas, o que poderia causar falhas na tarefa. Ele também ajuda a prever picos de uso de recursos.

Target (O que queremos prever):

- vertical_scaling (Escalonamento Vertical):
- Esta coluna indica se o escalonamento vertical (ajuste automático de recursos) foi utilizado para a tarefa. Os valores podem ser:
    - VERTICAL_SCALING_OFF: Escalonamento automático não habilitado.
    - VERTICAL_SCALING_CONSTRAINED: Escalonamento automático habilitado com parâmetros/condições fornecidas pelo usuário.
    - VERTICAL_SCALING_FULLY_AUTOMATED: Escalonamento automático habilitado sem limites fornecidos pelo usuário.
- O escalonamento vertical é o objetivo final que queremos prever. Entender como as diferentes características da tarefa influenciam a necessidade de ajuste automático de recursos nos ajuda a desenvolver um modelo que pode prever de forma precisa quando e como o escalonamento vertical deve ser aplicado para otimizar o uso de recursos e manter a eficiência do sistema.

Os gráficos a seguir ilustram a distribuição dos dados das colunas selecionadas:

1. Histograma das Memórias (average_usage_memory, maximum_usage_memory, assigned_memory):

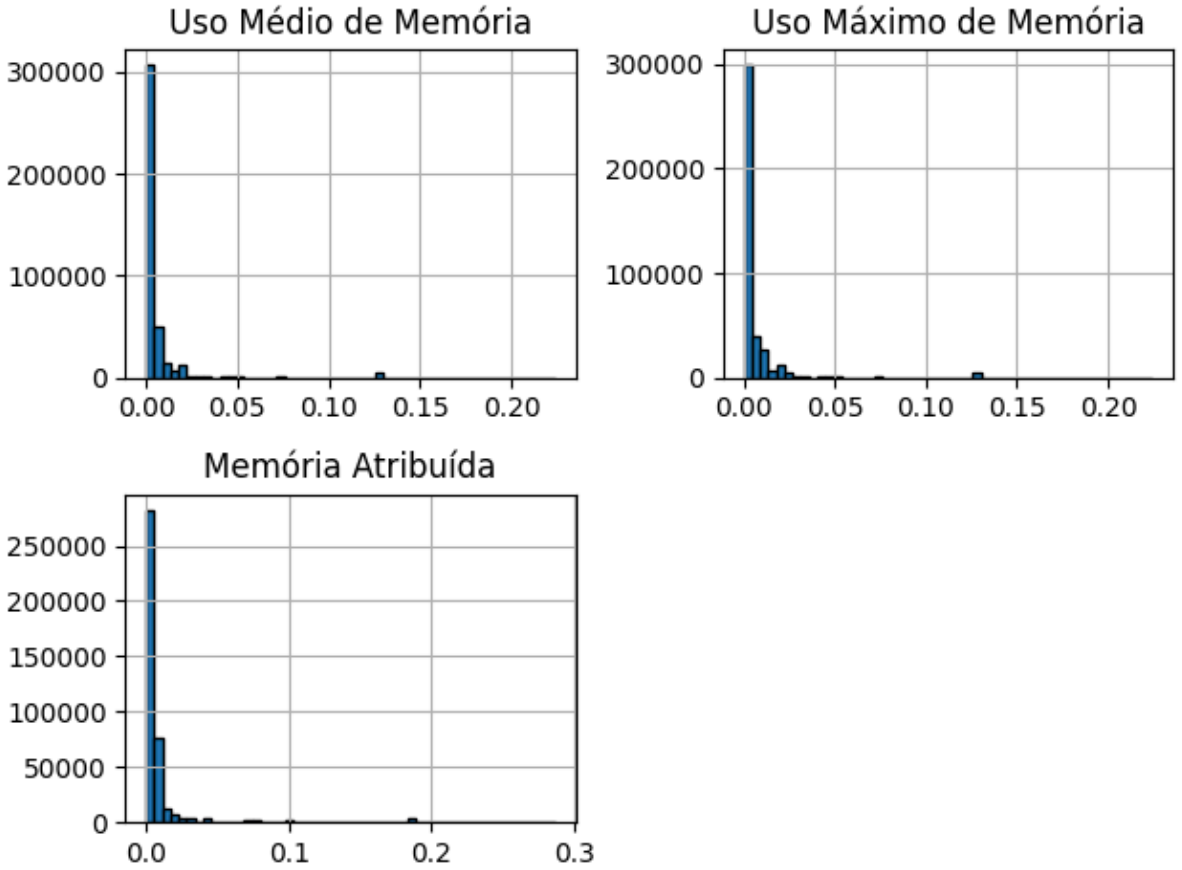

Figura 1: Distribuição das Features 'Uso Médio de Memória', 'Uso Máximo de Memória' e 'Memória Atribuída'. Este gráfico mostra a distribuição dos valores de uso de memória médio, máximo e alocado. Foi criado pelos autores deste artigo Google Colab **[36]**.

Os gráficos acima representam histogramas das colunas de memória que utilizaremos para treinamento do modelo, contendo as seguintes informações:

- Eixo X: Intervalo de valores (bins) para cada feature.
  - Uso Médio de Memória: Valor médio de memória utilizada ao longo do tempo. Unidade: Megabytes (MB).
  - Uso Máximo de Memória: Maior valor de memória utilizado em qualquer momento específico. Unidade: Megabytes (MB).
  - Memória Atribuída: Quantidade de memória alocada para a aplicação. Unidade: Megabytes (MB).

- Eixo Y: Frequência ou contagem de ocorrências para cada faixa de valores.

O gráfico de distribuição das features mostra a distribuição dos valores de Uso Médio de Memória, Uso Máximo de Memória e Memória Atribuída. Esse gráfico é importante porque nos permite visualizar a variabilidade e a dispersão dos dados. Nota-se a concentração em faixas específicas, o que nos indica padrões, e, nesse sentido, a capacidade do modelo entender o funcionamento e alocar os recursos necessários. Além disso, a comparação com a Memória Atribuída pode revelar se a memória designada é suficiente para o uso médio e máximo observado. Os histogramas acima mostram uma grande concentração de valores próximos a zero, indicando que a maioria dos dados de memória está em uma faixa muito baixa.

2. Boxplots das Memórias (average_usage_memory, maximum_usage_memory, assigned_memory):

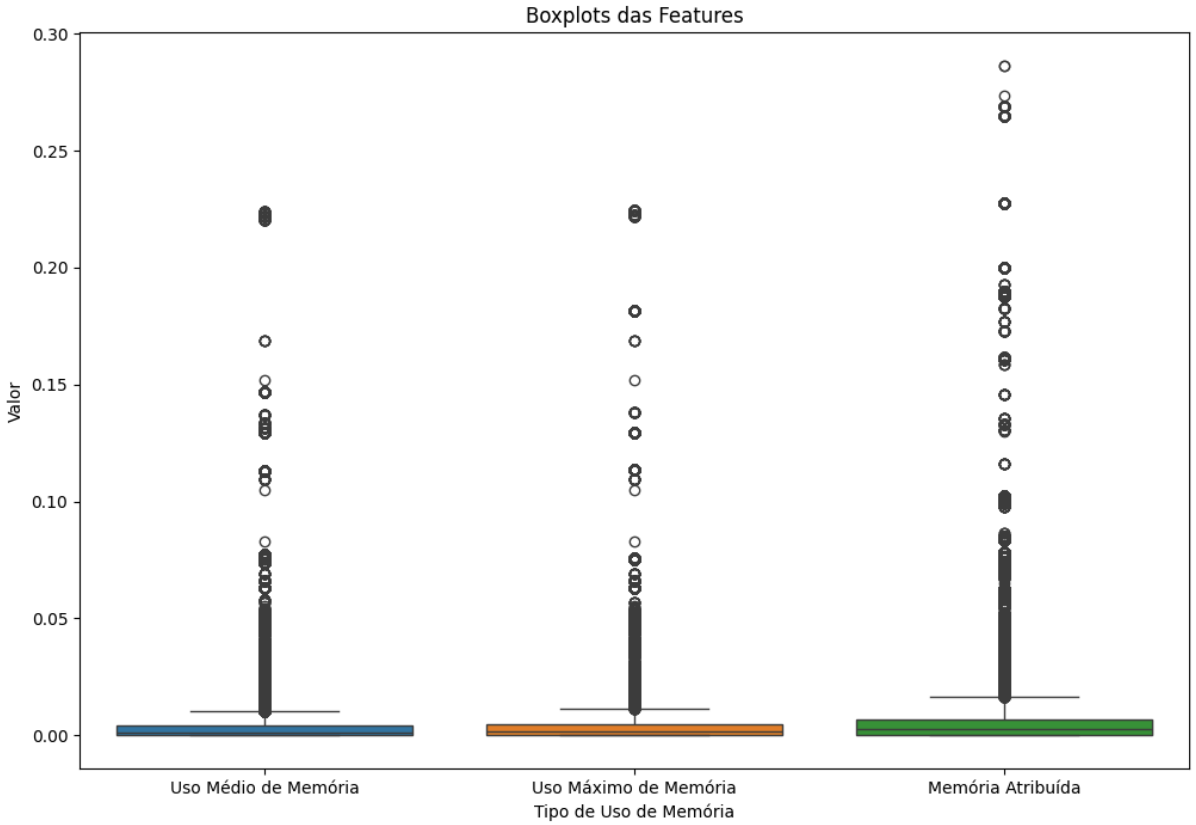

Figura 2: Boxplots das Features 'Uso Médio de Memória', 'Uso Máximo de Memória' e 'Memória Atribuída'. Este gráfico compara a distribuição dessas três variáveis.

Foi criado pelos autores deste artigo no Google Colab **[36]**.

Os boxplots das features mostram a distribuição dos dados e destacam a mediana, os quartis e os outliers para Uso Médio de Memória, Uso Máximo de Memória e Memória Atribuída. Cada eixo desse gráfico contém as seguintes informações:

- Eixo X: Tipos de uso de memória
  - Uso Médio de Memória. Unidade: Megabytes (MB).
  - Uso Máximo de Memória. Unidade: Megabytes (MB).
  - Memória Atribuída. Unidade: Megabytes (MB).
- Eixo Y: Valores de memória (número contínuo).

Esse gráfico é importante para identificar outliers e a distribuição dos dados. Outliers podem influenciar significativamente o desempenho dos modelos de machine learning. Os boxplots evidenciam a presença de outliers, especialmente nas colunas da base de dados “maximum_usage_memory” e “assigned_memory”, sugerindo uma variação significativa em alguns casos, que mostra a eventual necessidade de tratamento dos mesmos.

3. Boxplot Comparação de Average Usage Memory entre Eventos:

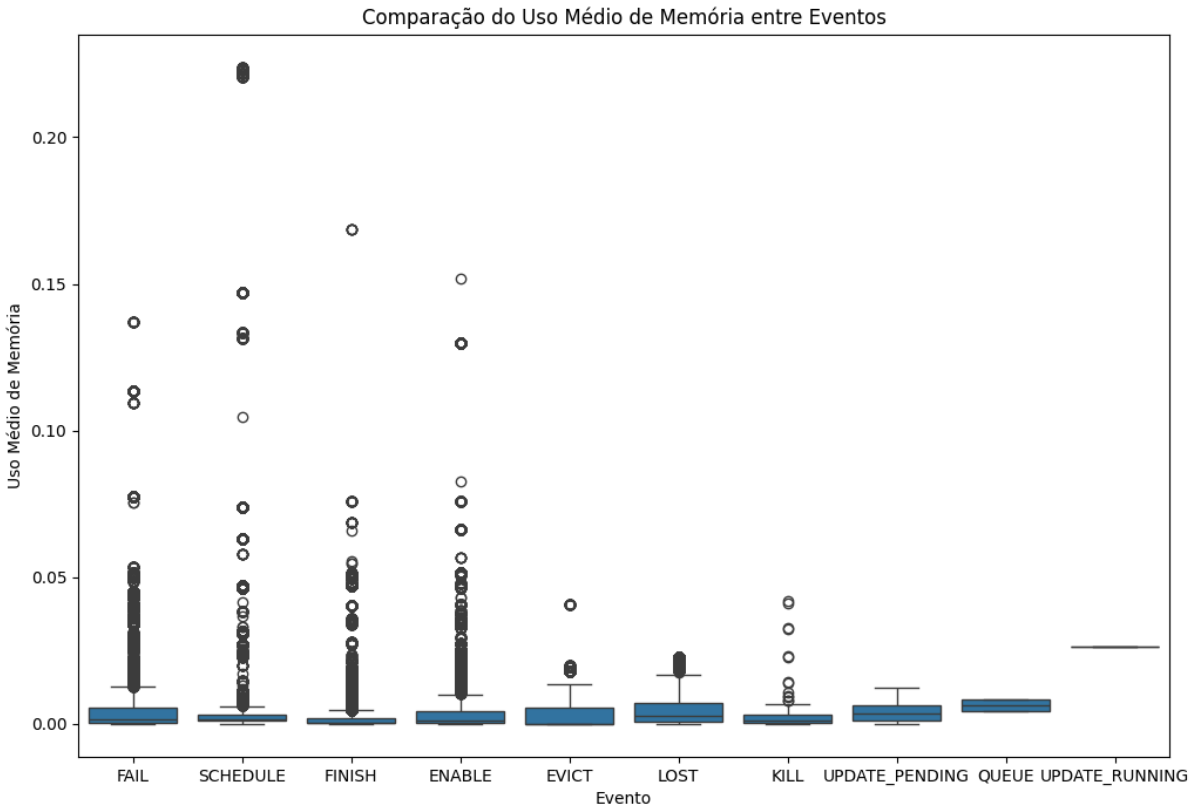

Figura 3: Comparação da 'Uso Médio de Memória' entre Eventos. Este gráfico compara o uso médio de memória entre diferentes tipos de eventos.
Foi criado pelos autores deste artigo no Google Colab [36].

Este gráfico compara a coluna da base de dados “average_usage_memory” entre diferentes tipos de eventos, como FAIL, SCHEDULE, FINISH, etc. Os dois eixos do gráfico contém as seguintes informações:

- Eixo X: Tipo de evento (Evento).
- Eixo Y: Uso Médio de Memória. Unidade: Megabytes (MB).

O mais importante de se observar nesse gráfico é a diferença entre eventos, verificando como o uso de memória varia entre diferentes tipos de eventos, alguns deles mostram uso médio de memória mais alto que outros. Por exemplo, eventos como SUBMIT ou FINISH podem ter padrões de uso de memória distintos. Esses nos ajudaram no desenvolvimento dos modelos de IA, pois nos permitirão ajustar as previsões de acordo com o tipo de evento. No modelo Random Forest, essa diferenciação pode ajudar a prever com maior precisão o estado da aplicação sob diferentes condições. No modelo LSTM, essas informações podem ser incorporadas nas sequências temporais para melhorar a previsão de picos de carga.

Para análise do rendimento de nosso modelo utilizaremos três métricas:

- MSE, representa a média da diferença entre o valor original e o valor previsto pelo modelo, elevado ao quadrado, ele mede a variância dos resíduos [37].

$$MSE\left(y, \hat{y}\right) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2$$

Figura 4: Equação do erro quadrático médio [38].

- MAE, erro médio absoluto, representa a média da diferença absoluta entre o valor atual e o valor previsto pelo modelo, ele mede a média dos resíduos na base de dados. Quanto menor o valor do MAE, melhor são os resultados previstos pelo nosso modelo [37] [38].

$$MAE\left(y, \hat{y}\right) = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \hat{y}_i\right|$$

Figura 5: Equação do erro médio absoluto [38].

- R^2 ou Coeficiente de Determinação, representa a proporção da variância da variável dependente que é explicada pelo modelo. É uma pontuação sem escala, ou seja, independente dos valores serem pequenos ou grandes, o valor de R ao quadrado será menor que um. Quanto maior é o valor de R^2 (mais próximo de 1), o modelo é mais explicativo em relação aos dados que foram previstos, quanto mais próximo de 0, pior o modelo **[37]** **[38]**.

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Figura 6: Equação do coeficiente de determinação **[38]**.

- y, valores reais;
- ŷ, valores preditos pelo modelo;
- |y - ŷ|, módulo da diferença entre y e ŷ;

Codificação do Primeiro Modelo:

1. Importação das Bibliotecas **[39]**

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import joblib
```

2. Carregar o DataSet

```
# Carregar o DataSet
google = pd.read_csv('datasets/google.csv')
```

3. Pré-processamento dos Dados

```
# Pré-processamento do Google Dataset
google['time'] = pd.to_datetime(google['time'], unit='ns')
google.set_index('time', inplace=True)

# Preencher valores ausentes apenas nas colunas numéricas
numeric_cols = google.select_dtypes(include=[np.number]).columns
google[numeric_cols] = google[numeric_cols].fillna(google[numeric_cols].mean())
```

4. Extração do Valores de Memória

```
def extract_memory_value(value):
    if isinstance(value, str):
        value_dict = eval(value)
        return value_dict.get('memory', np.nan)
    return value


google['average_usage_memory'] = google['average_usage'].apply(extract_memory_value)
google['maximum_usage_memory'] = google['maximum_usage'].apply(extract_memory_value)
```

Este trecho de código converte strings contendo informações de uso de memória em valores numéricos.

5. Seleção de Features e Target para o Modelo de Árvores de Decisão
6. Divisão dos Dados em Conjuntos de Treinamento e Teste
7. Criação e Treinamento do Modelo de Árvore de Decisão

```
# Selecionar features e target para o modelo de árvore de decisão
X_google = google[['scheduling_class', 'priority', 'assigned_memory',
'average_usage_memory', 'maximum_usage_memory']]
y_google = google['vertical_scaling']


# Dividir os dados em conjuntos de treinamento e teste
X_train_google, X_test_google, y_train_google, y_test_google =
train_test_split(X_google, y_google, test_size=0.2, random_state=42)

# Criar e treinar o modelo de árvore de decisão
dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train_google, y_train_google)
```

8. Avaliação do Modelo

```
# Avaliar o modelo
y_pred_google = dt_model.predict(X_test_google)
mse_google = mean_squared_error(y_test_google, y_pred_google)
mae_google = mean_absolute_error(y_test_google, y_pred_google)
r2_google = r2_score(y_test_google, y_pred_google)


print(f'MSE do modelo de árvore de decisão: {mse_google}')
print(f'MAE do modelo de árvore de decisão: {mae_google}')
print(f'R² do modelo de árvore de decisão: {r2_google}')
```

9. Visualização da Importância das Features

```
# Visualizar a importância das features
importances = dt_model.feature_importances_
feature_names = X_google.columns
feature_importances = pd.Series(importances,
index=feature_names).sort_values(ascending=False)

print("Importância das features:\n", feature_importances)
```

10. Salvar o Modelo de Árvore de Decisão

```
joblib.dump(dt_model, 'decision_tree_model.pkl')
```

Testes do Modelo

Os testes do modelo de Árvore de Decisão foram realizados com as seguintes saídas:

```
python test_decision_model.py
MSE do modelo de árvore de decisão: 0.0008376473195578182
MAE do modelo de árvore de decisão: 0.000649631561578794
R² do modelo de árvore de decisão: 0.9985323594431326
Importância das features:
 priority                0.369350
assigned_memory         0.245117
scheduling_class        0.236193
maximum_usage_memory    0.077163
average_usage_memory    0.072177
dtype: float64

Testando aplicações saudáveis:
A aplicação 0 está em estado saudável: 3.0
A aplicação 1 está em estado saudável: 2.0
A aplicação 2 está em estado saudável: 3.0

Testando aplicações com falha:
A aplicação 0 está em risco: 1.0
A aplicação 1 está em estado saudável: 2.0
A aplicação 2 está em estado saudável: 2.0
```

Métricas Iniciais

As métricas iniciais mostraram um ótimo ajuste aos dados de treinamento, mas levantaram suspeitas de overfitting:

- MSE (Treinamento): 3.942905195610498e-34
- MAE (Treinamento): 5.54709430033412e-19
- R^2 (Treinamento): 1.0
- MSE (Teste): 0.0016600758876243423
- MAE (Teste): 0.001184662887971001

- R^2 (Teste): 0.9970635966651407

A saída do modelo indica que ele tem um MSE e MAE muito baixos, e um R^2 muito alto, sugerindo que o modelo se ajusta muito bem aos dados de treinamento. No entanto, isso pode indicar um problema de overfitting, onde o modelo se ajusta muito bem aos dados de treinamento, mas pode não generalizar bem para novos dados.

Um R^2 muito alto nos dados de treinamento e uma discrepância significativa entre as métricas de treinamento e teste indicaram overfitting. Para mitigar isso, optamos por usar Random Forest, que é mais robusto e pode reduzir o risco de overfitting **[37]**.

Ajustes e Implementação do Random Forest

Para mitigar o overfitting, ajustamos o modelo para incluir Random Forest **[40]**:

1. Seleção de features: Adicionamos “resource_request_memory”, “cycles_per_instruction”, e “memory_accesses_per_instruction” como novas features.

cycles_per_instruction (Ciclos por Instrução):

- Esta coluna mede o número médio de ciclos de clock da CPU necessários para executar uma instrução. É uma métrica de desempenho que ajuda a entender a eficiência das tarefas em termos de uso de CPU. Tarefas com valores altos podem estar causando gargalos e podem precisar de otimização.
- Baixo valor: Indica que a CPU está executando instruções de maneira eficiente, gastando menos ciclos por instrução.
- Alto valor: Pode indicar ineficiências, como espera por dados de memória (latência de memória), ou execução de instruções complexas que requerem mais ciclos.

memory_accesses_per_instruction (Acessos à Memória por Instrução)

- Esta coluna mede o número médio de acessos à memória realizados por instrução. Isso inclui leituras e escritas na memória principal (RAM). Isso ajuda a entender o padrão de acesso à memória das tarefas. Tarefas que acessam muita memória podem precisar de otimizações específicas, como cache ou melhorias na eficiência do código.

- Baixo valor: Indica que a tarefa não depende muito de acessos à memória e pode estar executando principalmente operações computacionais.
- Alto valor: Indica que a tarefa faz muitos acessos à memória, o que pode causar latências se a memória for um gargalo.

2. Divisão dos dados: Usamos TimeSeriesSplit para criar divisões de treinamento e teste, respeitando a sequência temporal dos dados.
3. Modelo: Substituímos DecisionTreeRegressor por RandomForestRegressor para aumentar a robustez e reduzir o overfitting **[41]**.

```
# Selecionar features e target para o modelo de árvore de decisão
features = ['scheduling_class', 'priority', 'assigned_memory', 'average_usage_memory',
'maximum_usage_memory', 'resource_request_memory', 'cycles_per_instruction',
'memory_accesses_per_instruction']
X_google = google[features]
y_google = google['vertical_scaling']

# Dividir os dados em conjuntos de treinamento e teste usando TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
train_index, test_index = next(tscv.split(X_google))

X_train_google, X_test_google = X_google.iloc[train_index], X_google.iloc[test_index]
y_train_google, y_test_google = y_google.iloc[train_index], y_google.iloc[test_index]

# Criar e treinar o modelo de Random Forest para mitigar o overfitting
rf_model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
rf_model.fit(X_train_google, y_train_google)


# Avaliar o modelo nos dados de teste
y_pred_google = rf_model.predict(X_test_google)
mse_google = mean_squared_error(y_test_google, y_pred_google)
mae_google = mean_absolute_error(y_test_google, y_pred_google)
r2_google = r2_score(y_test_google, y_pred_google)

print(f'MSE do modelo Random Forest (teste): {mse_google}')
print(f'MAE do modelo Random Forest (teste): {mae_google}')
print(f'R² do modelo Random Forest (teste): {r2_google}')


# Avaliar o modelo nos dados de treinamento
y_train_pred_google = rf_model.predict(X_train_google)
mse_train_google = mean_squared_error(y_train_google, y_train_pred_google)
mae_train_google = mean_absolute_error(y_train_google, y_train_pred_google)
r2_train_google = r2_score(y_train_google, y_train_pred_google)
print(f'MSE do modelo Random Forest (treinamento): {mse_train_google}')
print(f'MAE do modelo Random Forest (treinamento): {mae_train_google}')
print(f'R² do modelo Random Forest (treinamento): {r2_train_google}')


# Realizar validação cruzada
cv_scores = cross_val_score(rf_model, X_google, y_google, cv=tscv, scoring='r2')
print(f'R² médio na validação cruzada: {cv_scores.mean()}')
```

```
# Visualizar a importância das features
importances = rf_model.feature_importances_
feature_names = X_google.columns
feature_importances = pd.Series(importances,
index=feature_names).sort_values(ascending=False)

print("Importância das features:\n", feature_importances)

# Salvar o modelo de Random Forest
joblib.dump(rf_model, 'random_forest_model.pkl')
```

Os novos resultados mostraram uma melhoria na generalização e redução do overfitting:

- MSE (Treinamento): 0.03354489138702923
- MAE (Treinamento): 0.060125362934210065
- R^2 (Treinamento): 0.9407692212128994
- MSE (Teste): 0.035924240848401484
- MAE (Teste): 0.061741364127223405
- R^2 (Teste): 0.9368855111572303
- R^2 Médio na Validação Cruzada: 0.9375932873658026

3.3 Testes e Simulação de Aplicação: Random Forest

A simulação consiste em dois componentes principais:

1. **app.py**: Representa uma aplicação web que analisa constantemente seu estado e faz previsões sobre a necessidade de alocação de recursos.
2. **stress_test.py**: Simula picos de carga para testar a robustez da aplicação web.

Código da Aplicação Web (app.py)

O app.py implementa uma aplicação Flask que prevê o estado da aplicação web a cada dois segundos, simulando um aumento de carga. Ele usa um modelo de Random Forest treinado para essa finalidade.

```
from flask import Flask, request, jsonify
import joblib
import pandas as pd
import threading
import time
import numpy as np

app = Flask(__name__)

# Carregar o modelo salvo
```

```
model = joblib.load('random_forest_model.pkl')

google = pd.read_csv('datasets/google.csv')
google['time'] = pd.to_datetime(google['time'], unit='ns')
google.set_index('time', inplace=True)

# Preencher valores ausentes apenas nas colunas numéricas
numeric_cols = google.select_dtypes(include=[np.number]).columns
google[numeric_cols] = google[numeric_cols].fillna(google[numeric_cols].mean())

# Extrair valores de memória dos dicionários em average_usage e maximum_usage
def extract_memory_value(value):
    if isinstance(value, str):
        value_dict = eval(value)
        return value_dict.get('memory', np.nan)
    return value

google['average_usage_memory'] = google['average_usage'].apply(extract_memory_value)
google['maximum_usage_memory'] = google['maximum_usage'].apply(extract_memory_value)
google['resource_request_memory'] =
google['resource_request'].apply(extract_memory_value)


# Selecionar um exemplo de sucesso (aplicação saudável) como estado atual
current_state = google[google['event'] == 'FINISH'][['scheduling_class', 'priority',
'assigned_memory', 'average_usage_memory', 'maximum_usage_memory',
'resource_request_memory', 'cycles_per_instruction',
'memory_accesses_per_instruction']].head(1).to_dict('records')[0]

# Função para prever o estado da aplicação
def predict_application_state(input_data):
    prediction = model.predict(input_data)
    return prediction

# Função para atualizar o estado da aplicação e enviar relatório
def update_state_and_report():
    global current_state
    while True:
        input_data = pd.DataFrame([current_state])
        prediction = predict_application_state(input_data)
        state_message = "Estado desconhecido"

        if prediction[0] < 1:
            state_message = 'A aplicação precisa de mais recursos:
{:.2f}'.format(prediction[0])
        elif prediction[0] < 2:
            state_message = 'A aplicação está em risco: {:.2f}'.format(prediction[0])
        else:
            state_message = 'A aplicação está em estado saudável:
{:.2f}'.format(prediction[0])

        print(state_message)
        # Simulando aumento de carga
        current_state["average_usage_memory"] += 0.1
        current_state["maximum_usage_memory"] += 0.1

        # Enviar relatório a cada 2 segundos
        time.sleep(2)

@app.route('/predict', methods=['POST'])
def predict():
    global current_state
```

```
    data = request.json
    input_data = pd.DataFrame(data)
    prediction = predict_application_state(input_data)
    response = []
    for pred in prediction:
        if pred < 1:
            response.append('A aplicação precisa de mais recursos: {:.2f}'.format(pred))
        elif pred < 2:
            response.append('A aplicação está em risco: {:.2f}'.format(pred))
        else:
            response.append('A aplicação está em estado saudável: {:.2f}'.format(pred))
    return jsonify(response)

if __name__ == '__main__':
    # Iniciar a thread para atualizar o estado e enviar relatórios
    threading.Thread(target=update_state_and_report, daemon=True).start()
    app.run(host='0.0.0.0', port=5000)
```

Simulação de Picos de Carga (stress_test.py)

O **stress_test.py** simula um pico de carga enviando múltiplas requisições simultâneas para a aplicação Flask.

```
import requests
import json
from concurrent.futures import ThreadPoolExecutor
import pandas as pd
import numpy as np

# Carregar o dataset completo para extrair exemplos reais
google = pd.read_csv('datasets/google.csv')
google['time'] = pd.to_datetime(google['time'], unit='ns')
google.set_index('time', inplace=True)

# Preencher valores ausentes apenas nas colunas numéricas
numeric_cols = google.select_dtypes(include([np.number]).columns)
google[numeric_cols] = google[numeric_cols].fillna(google[numeric_cols].mean())

# Extrair valores de memória dos dicionários em average_usage e maximum_usage
def extract_memory_value(value):
    if isinstance(value, str):
        value_dict = eval(value)
        return value_dict.get('memory', np.nan)
    return value

google['average_usage_memory'] = google['average_usage'].apply(extract_memory_value)
google['maximum_usage_memory'] = google['maximum_usage'].apply(extract_memory_value)
google['resource_request_memory'] =
google['resource_request'].apply(extract_memory_value)

# Selecionar exemplos de falha para o teste de estresse
test_data_fail = google[google['event'] == 'FAIL'][['scheduling_class', 'priority',
'assigned_memory', 'average_usage_memory', 'maximum_usage_memory',
'resource_request_memory', 'cycles_per_instruction',
'memory_accesses_per_instruction']].head(3).to_dict('records')
```

```python
# URL da aplicação Flask
url = 'http://localhost:5000/predict'

# Função para enviar uma requisição
def send_request(data):
    response = requests.post(url, headers={"Content-Type": "application/json"},
data=json.dumps([data]))
    print(response.json())

# Número de requisições simultâneas
num_requests = 100

# Usar ThreadPoolExecutor para enviar requisições simultâneas
with ThreadPoolExecutor(max_workers=num_requests) as executor:
    futures = [executor.submit(send_request, data) for data in test_data_fail for _ in
range(num_requests // len(test_data_fail))]

# Esperar todas as requisições completarem
for future in futures:
    future.result()
```

Resultados da Simulação

Saída de **app.py**:

```
A aplicação está em risco: 1.93
A aplicação está em risco: 1.93
A aplicação está em risco: 1.93
A aplicação está em risco: 1.93

127.0.0.1 - - [06/Jun/2024 14:25:41] "POST /predict HTTP/1.1" 200 -
```

Saída de **stress_test.py**:

```
['A aplicação está em risco: 1.10']
['A aplicação está em risco: 1.10']
['A aplicação está em risco: 1.10']
['A aplicação está em risco: 2.00']
```

Análise dos Resultados

O modelo de Random Forest foi capaz de prever o estado da aplicação com base nas métricas fornecidas. Durante o teste de estresse, a aplicação repetidamente indicou que estava em risco, conforme previsto pelo modelo. Este comportamento é esperado quando a carga na aplicação aumenta, simulando um pico de carga. O uso de múltiplas requisições simultâneas no **stress_test.py** foi eficaz para testar a robustez do modelo e da aplicação.

A principal diferença entre os modelos é a robustez e a capacidade de generalização. Enquanto a Árvore de Decisão mostrou-se eficiente, o Random Forest reduziu significativamente o overfitting, conforme evidenciado pelas métricas mais balanceadas entre os dados de treinamento e teste.

- Árvore de Decisão: Excelente ajuste nos dados de treinamento, mas suscetível a overfitting. Bom para uma análise inicial e rápida implementação.
- Random Forest: Melhor generalização, menor overfitting, e resultados mais robustos em situações de pico de carga.

Durante os testes de estresse, ambos os modelos identificaram corretamente quando a aplicação estava em risco. No entanto, o Random Forest forneceu previsões mais consistentes e resilientes sob carga pesada, confirmando sua superioridade em cenários de alta variabilidade e complexidade dos dados.

Em resumo, para a tarefa de prever a necessidade de alocação de recursos em uma aplicação web, o Random Forest provou ser uma escolha mais confiável, enquanto a Árvore de Decisão serviu como uma base eficaz para a construção inicial do modelo. Este estudo demonstra a importância de escolher o modelo adequado para diferentes fases de desenvolvimento e tipos de dados, garantindo previsões precisas e confiáveis para a gestão de recursos em ambientes distribuídos.

3.4 Desenvolvimento do Segundo Modelo: LSTM

Agora, vamos adaptar a nossa aplicação web Flask para modelo de LSTM, que foi construído para prever picos de carga. Utilizando as mesmas técnicas e metodologias de carregamento e pré-processamento dos dados que foram utilizadas para construir o modelo de Random Forest:

- Os dados do Google Cluster foram carregados e pré-processados.
- As colunas numéricas foram preenchidas com a média dos valores ausentes.
- Os valores de memória foram extraídos de dicionários em colunas específicas.

A principal diferença entre a construção dos dois modelos é a configuração do modelo de LSTM:

- O modelo foi construído usando Keras com TensorFlow como backend.

- Compilação do modelo com o otimizador 'adam' e a função de perda 'mean_squared_error'.
- Os dados foram escalados e divididos em sequências para treinar o modelo.

Código do Segundo Modelo:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.callbacks import EarlyStopping
import joblib

# Carregar o dataset
google = pd.read_csv('datasets/google.csv')

# Pré-processamento do Google Dataset
google['time'] = pd.to_datetime(google['time'], unit='ns')
google.set_index('time', inplace=True)

# Preencher valores ausentes apenas nas colunas numéricas
numeric_cols = google.select_dtypes(include=[np.number]).columns
google[numeric_cols] = google[numeric_cols].fillna(google[numeric_cols].mean())

# Extrair valores de memória dos dicionários em average_usage e maximum_usage
def extract_memory_value(value):
    if isinstance(value, str):
        value_dict = eval(value)
        return value_dict.get('memory', np.nan)
    return value

google['average_usage_memory'] = google['average_usage'].apply(extract_memory_value)
google['maximum_usage_memory'] = google['maximum_usage'].apply(extract_memory_value)
google['resource_request_memory'] =
google['resource_request'].apply(extract_memory_value)

# Selecionar as features relevantes
features = ['scheduling_class', 'priority', 'assigned_memory', 'average_usage_memory',
'maximum_usage_memory', 'resource_request_memory', 'cycles_per_instruction',
'memory_accesses_per_instruction']
X_google = google[features]
y_google = google['vertical_scaling']

# Remover quaisquer linhas com NaN em X_google ou y_google
data = pd.concat([X_google, y_google], axis=1).dropna()
X_google = data[features]
y_google = data['vertical_scaling']

# Normalizar os dados
scaler = StandardScaler()
X_google_scaled = scaler.fit_transform(X_google)

# Criar janelas de dados para o LSTM
def create_sequences(data, window_size):
    sequences = []
    for i in range(len(data) - window_size):
        sequences.append(data[i:i + window_size])
```

```
    return np.array(sequences)

window_size = 10
X_sequences = create_sequences(X_google_scaled, window_size)
y_sequences = y_google[window_size:].values

# Dividir os dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X_sequences, y_sequences,
test_size=0.2, random_state=42)

# Reshape para [amostras, janelas de tempo, features]
X_train = X_train.reshape((X_train.shape[0], window_size, X_google.shape[1]))
X_test = X_test.reshape((X_test.shape[0], window_size, X_google.shape[1]))

# Construir o modelo LSTM
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(window_size, X_google.shape[1])))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')

# Treinar o modelo
early_stopping = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2,
callbacks=[early_stopping])

# Avaliar o modelo
y_pred = model.predict(X_test)

# Verificar NaNs em y_pred
if np.isnan(y_pred).any():
    print("y_pred contém NaNs. Ajuste necessário nos dados ou no modelo.")
else:
    mse = mean_squared_error(y_test, y_pred)
    print(f'MSE do modelo LSTM: {mse}')

# Salvar o modelo
model.save('lstm_model.h5')
joblib.dump(scaler, 'scaler.pkl')


# Função para prever picos de carga
def predict_traffic_peak(input_data):
    from tensorflow.keras.models import load_model
    model = load_model('lstm_model.h5', compile=False)
    model.compile(optimizer='adam', loss='mean_squared_error')
    scaler = joblib.load('scaler.pkl')
    input_data_scaled = scaler.transform(input_data)
    input_data_sequences = create_sequences(input_data_scaled, window_size)
    if input_data_sequences.shape[0] == 0:
        raise ValueError("Os dados de entrada são insuficientes para criar sequências
com o tamanho da janela especificado.")
    input_data_sequences = input_data_sequences.reshape((input_data_sequences.shape[0],
window_size, X_google.shape[1]))
    prediction = model.predict(input_data_sequences)
    return prediction

# Exemplo de uso da função de previsão
new_data = pd.DataFrame({
    'scheduling_class': [2]*15,
    'priority': [0]*15,
    'assigned_memory': [0.5]*15,
```

```
    'average_usage_memory': [4.0]*15,
    'maximum_usage_memory': [5.0]*15,
    'resource_request_memory': [3.0]*15,
    'cycles_per_instruction': [2.0]*15,
    'memory_accesses_per_instruction': [1.5]*15
})

try:
    peak_prediction = predict_traffic_peak(new_data)
    print(f'Previsão de pico de carga: {peak_prediction[0]}')
except ValueError as e:
    print(f"Erro: {e}")
```

3.5 Testes e Simulação de Aplicação: LSTM

A simulação será feita da mesma forma que foi conduzida pelo primeiro modelo:

- app.py: Refatorando o código dessa aplicação web genérica que agora, além de analisar o estado da aplicação, também fará uma previsão se a mesma sofrerá um pico de carga
- stress_test.py: Permanece o mesmo, será o código que provocará um pico de carga.

Código da Aplicação Web (app.py) Refatorado

```
from flask import Flask, request, jsonify
import joblib
import pandas as pd
import threading
import time
import numpy as np
from tensorflow.keras.models import load_model

app = Flask(__name__)

# Carregar os modelos salvos
rf_model = joblib.load('random_forest_model.pkl')
lstm_model = load_model('lstm_model.h5', compile=False)
lstm_model.compile(optimizer='adam', loss='mean_squared_error')  # Certifique-se de
compilar o modelo com a função de perda correta
scaler = joblib.load('scaler.pkl')

# Carregar o dataset completo para extrair exemplos reais
google = pd.read_csv('datasets/google.csv')
google['time'] = pd.to_datetime(google['time'], unit='ns')
google.set_index('time', inplace=True)

# Preencher valores ausentes apenas nas colunas numéricas
numeric_cols = google.select_dtypes(include=[np.number]).columns
google[numeric_cols] = google[numeric_cols].fillna(google[numeric_cols].mean())

# Extrair valores de memória dos dicionários em average_usage e maximum_usage
def extract_memory_value(value):
    if isinstance(value, str):
        value_dict = eval(value)
        return value_dict.get('memory', np.nan)
    return value

google['average_usage_memory'] = google['average_usage'].apply(extract_memory_value)
google['maximum_usage_memory'] = google['maximum_usage'].apply(extract_memory_value)
google['resource_request_memory'] =
google['resource_request'].apply(extract_memory_value)

# Selecionar um exemplo de sucesso (aplicação saudável) como estado atual
current_state = google[google['event'] == 'FINISH'][['scheduling_class', 'priority',
'assigned_memory', 'average_usage_memory', 'maximum_usage_memory',
'resource_request_memory', 'cycles_per_instruction',
'memory_accesses_per_instruction']].head(1).to_dict('records')[0]

# Tamanho da janela para o LSTM
window_size = 10
historical_data = []

# Função para prever o estado da aplicação
def predict_application_state(input_data):
    prediction = rf_model.predict(input_data)
    return prediction

# Função para criar sequências
def create_sequences(data, window_size):
    sequences = []
    for i in range(len(data) - window_size + 1):
        sequences.append(data[i:i + window_size])
```

```
    return np.array(sequences)

# Função para prever picos de carga
def predict_traffic_peak(input_data):
    input_data_scaled = scaler.transform(input_data)
    input_data_sequences = create_sequences(input_data_scaled, window_size)
    if input_data_sequences.shape[0] == 0:
        raise ValueError("Os dados de entrada são insuficientes para criar sequências
com o tamanho da janela especificado.")
    input_data_sequences = input_data_sequences.reshape((input_data_sequences.shape[0],
window_size, input_data.shape[1]))
    prediction = lstm_model.predict(input_data_sequences)
    return prediction

# Função para atualizar o estado da aplicação e enviar relatório
def update_state_and_report():
    global current_state, historical_data
    while True:
        input_data = pd.DataFrame([current_state])
        state_prediction = predict_application_state(input_data)
        historical_data.append(current_state)

        # Manter apenas os últimos `window_size` registros
        if len(historical_data) > window_size:
            historical_data.pop(0)

        state_message = "Estado desconhecido"
        peak_message = "Previsão desconhecida"

        if state_prediction[0] < 1:
            state_message = 'A aplicação precisa de mais recursos:
{:.2f}'.format(state_prediction[0])
        elif state_prediction[0] < 2:
            state_message = 'A aplicação está em risco:
{:.2f}'.format(state_prediction[0])
        else:
            state_message = 'A aplicação está em estado saudável:
{:.2f}'.format(state_prediction[0])

        # Prever pico de carga somente se tivermos dados suficientes
        if len(historical_data) == window_size:
            historical_df = pd.DataFrame(historical_data)
            try:
                peak_prediction = predict_traffic_peak(historical_df)
                if peak_prediction[0][0] < 1:
                    peak_message = 'A aplicação não terá pico de carga iminente'
                else:
                    peak_message = 'A aplicação terá um pico de carga:
{:.2f}'.format(peak_prediction[0][0])
            except Exception as e:
                peak_message = str(e)

        print(state_message)
        print(peak_message)

        # Simulando aumento de carga
        current_state["average_usage_memory"] += 0.1
        current_state["maximum_usage_memory"] += 0.1

        # Enviar relatório a cada 2 segundos
        time.sleep(2)
```

```
@app.route('/predict', methods=['POST'])
def predict():
    global current_state, historical_data
    data = request.json
    input_data = pd.DataFrame(data)
    state_prediction = predict_application_state(input_data)

    # Adicionar os novos dados históricos
    historical_data.append(input_data.iloc[0].to_dict())
    if len(historical_data) > window_size:
        historical_data.pop(0)

    response = []
    for state_pred in state_prediction:
        if state_pred < 1:
            state_message = 'A aplicação precisa de mais recursos:
{:.2f}'.format(state_pred)
        elif state_pred < 2:
            state_message = 'A aplicação está em risco: {:.2f}'.format(state_pred)
        else:
            state_message = 'A aplicação está em estado saudável:
{:.2f}'.format(state_pred)

        # Prever pico de carga somente se tivermos dados suficientes
        if len(historical_data) == window_size:
            historical_df = pd.DataFrame(historical_data)
            try:
                peak_prediction = predict_traffic_peak(historical_df)
                if peak_prediction[0][0] < 1:
                    peak_message = 'A aplicação não terá pico de carga iminente'
                else:
                    peak_message = 'A aplicação terá um pico de carga:
{:.2f}'.format(peak_prediction[0][0])
            except Exception as e:
                peak_message = str(e)
        else:
            peak_message = 'Dados insuficientes para prever pico de carga'

        response.append({'state_message': state_message, 'peak_message': peak_message})
    return jsonify(response)

if __name__ == '__main__':
    # Iniciar a thread para atualizar o estado e enviar relatórios
    threading.Thread(target=update_state_and_report, daemon=True).start()
    app.run(host='0.0.0.0', port=5000)
```

Saída dos testes da aplicação com o código refatorado:

```
A aplicação está em estado saudável: 3.00
Previsão desconhecida
A aplicação terá um pico de carga: 3.07
A aplicação está em risco: 1.93
Previsão desconhecida
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step

A aplicação está em risco: 1.93
```

Simulação dos Picos de Carga (stress test.py)

```
[{'peak_message': 'Dados insuficientes para prever pico de carga', 'state_message': 'A
aplicação está em risco: 1.10'}]
[{'peak_message': 'Dados insuficientes para prever pico de carga', 'state_message': 'A
aplicação está em risco: 1.10'}]
[{'peak_message': 'A aplicação terá um pico de carga: 1.83', 'state_message': 'A aplicação
está em risco: 1.10'}]
[{'peak_message': 'A aplicação terá um pico de carga: 1.83', 'state_message': 'A aplicação
está em risco: 1.10'}]
[{'peak_message': 'A aplicação terá um pico de carga: 1.88', 'state_message': 'A aplicação
está em risco: 1.10'}]
```

Resultados da Aplicação

1. **Estado da Aplicação:**
   - A aplicação apresenta diferentes estados de saúde, variando entre "estado saudável" e "em risco".
   - A mensagem "A aplicação está em estado saudável: 3.00" indica que o modelo Random Forest classificou o estado atual como saudável.
   - A mensagem "A aplicação está em risco: 1.93" sugere que o estado da aplicação está se deteriorando e pode precisar de atenção.
2. **Pico de Carga:**
   - A previsão de picos de carga usa um modelo LSTM. Inicialmente, a mensagem é "Previsão desconhecida", o que indica que não havia dados suficientes para fazer uma previsão.
   - Quando suficientes dados históricos são acumulados, a previsão de picos de carga é realizada, como indicado pela mensagem "A aplicação terá um pico de carga: 3.07".

Testes de Estresse (Stress Test)

1. **Mensagens de Estado:**
   - Durante o stress test, o estado da aplicação foi consistentemente "em risco", com mensagens como "A aplicação está em risco: 1.10".
   - A consistência dessas mensagens sugere que o stress test está aplicando uma carga significativa, mantendo a aplicação sob risco constante.

2. **Previsão de Pico de Carga:**
    - No início dos testes, as mensagens indicam "Dados insuficientes para prever pico de carga", mostrando que o modelo LSTM precisa de uma quantidade mínima de dados históricos para fazer previsões.
    - Uma vez que dados suficientes foram acumulados, as mensagens mudam para prever picos de carga, como "A aplicação terá um pico de carga: 1.83".
    - A progressão das previsões (de 1.83 para 1.88) sugere um aumento gradual na previsão de picos de carga, provavelmente devido ao aumento da carga de trabalho durante os testes de estresse.

4 - Conclusão

A utilização de técnicas de Inteligência Artificial (IA) para prever e prevenir picos de carga em aplicações web e sistemas distribuídos é um campo de estudo promissor. O projeto em questão conseguiu empregar modelos de aprendizado de máquina, mais especificamente, possibilitando aplicar as informações tiradas das análises para uma melhoria em um ambiente de produção.

Utilizando um modelo Random Forest, a aplicação consegue determinar se está em um "estado saudável" ou "em risco", em tempo real. Por exemplo, a mensagem "A aplicação está em estado saudável: 3.00" indica uma classificação positiva do modelo, sugerindo que a aplicação está operando dentro dos parâmetros esperados. Em contraste, a mensagem "A aplicação está em risco: 1.93" alerta para um estado de degradação, indicando que a aplicação pode precisar de intervenção ou ajuste de recursos.

A aplicação também prevê picos de carga utilizando um modelo LSTM. Inicialmente, a previsão de picos de carga não foi possível devido à insuficiência de dados históricos, conforme indicado pela mensagem "Previsão desconhecida". No entanto, uma vez que dados históricos suficientes foram acumulados, a previsão tornou-se viável, como evidenciado pela mensagem "A aplicação terá um pico de carga: 3.07". Isso demonstra a importância de uma base de dados robusta para a precisão das previsões.

Os testes de estresse foram essenciais para avaliar a resiliência da aplicação sob condições de carga elevada. Durante esses testes, a aplicação manteve-se

consistentemente em um estado de "risco", conforme ilustrado pelas mensagens repetitivas "A aplicação está em risco: 1.10". Essa consistência sugere que os testes de estresse aplicaram uma carga significativa, colocando a aplicação em um estado de risco contínuo.

A previsão de picos de carga durante os testes de estresse também evoluiu. No início, a mensagem "Dados insuficientes para prever pico de carga" destacava a necessidade de mais dados para previsões precisas. Após a acumulação de dados históricos suficientes, a aplicação começou a prever picos de carga, com mensagens como "A aplicação terá um pico de carga: 1.83" e "A aplicação terá um pico de carga: 1.88". A progressão dessas previsões indica um aumento gradual na carga de trabalho durante os testes de estresse, o que é esperado em um cenário de carga crescente.

A análise dos resultados e dos testes de estresse leva a várias conclusões importantes. Primeiramente, a combinação dos modelos Random Forest e LSTM mostrou-se eficaz na previsão tanto do estado atual da aplicação quanto de picos de carga iminentes, desde que haja dados históricos suficientes. Esta capacidade de previsão permite uma gestão proativa da aplicação, identificando e mitigando problemas antes que afetem o desempenho.

Em segundo lugar, a aplicação demonstrou ser capaz de lidar com cargas normais de trabalho, mas entrou em estado de risco sob condições de estresse prolongado. Isso indica a necessidade de ajustes na alocação de recursos ou otimizações no código para melhorar a resiliência. A precisão das previsões de picos de carga é altamente dependente da quantidade e qualidade dos dados históricos, destacando a importância de um sistema de monitoramento contínuo para capturar dados em tempo real.

A utilização de IA para prever e prevenir picos de carga é uma abordagem poderosa que pode melhorar significativamente a resiliência e a eficiência de aplicações modernas. Este projeto demonstra que, com a implementação correta de modelos de aprendizado de máquina e uma base de dados robusta, é possível gerenciar proativamente o desempenho da aplicação, evitando sobrecargas e mantendo um serviço de alta qualidade. A adoção dessas práticas pode beneficiar não apenas o desempenho técnico, mas também a satisfação dos usuários finais, garantindo uma operação mais estável e confiável, mesmo sob condições de alta demanda.

Por fim, é clara a importância e a eficácia que os modelos preditivos têm na realização de previsões quando se trata de picos de carga. Não obstante, técnicas de treinamento supervisionados também possuem grande influência principalmente quando se trata de responder perguntas categóricas como feito pelos modelos de árvore de decisão e de floresta de decisão, sendo o segundo responsável por uma resposta mais fidedigna não sendo tão afetado por dados inconsistentes.

De maneira mais específica, utilizando um modelo Random Forest, a aplicação consegue determinar se está em um "estado saudável" ou "em risco", em tempo real, realizando uma classificação de maneira categórica e mais facilmente interpretável.

A aplicação também prevê picos de carga utilizando um modelo LSTM. Inicialmente, a previsão de picos de carga não foi possível devido à insuficiência de dados históricos, conforme indicado pela mensagem "Previsão desconhecida".

No entanto, uma vez que dados históricos suficientes foram acumulados, a previsão tornou-se viável, como evidenciado pela mensagem "A aplicação terá um pico de carga: 3.07". Isso demonstra a importância de uma base de dados robusta para a precisão das previsões.

Os testes de estresse foram essenciais para avaliar a resiliência da aplicação sob condições de carga elevada. Durante esses testes, a aplicação manteve-se consistentemente em um estado de "risco". Essa consistência sugere que os testes de estresse aplicaram uma carga significativa, colocando a aplicação em um estado de risco contínuo.

Assim, o objetivo inicial do trabalho foi alcançado, em que conseguiu-se criar um modelo preditivo baseado na análise de séries temporais para a possível melhora de eficiência de um sistema.

No entanto, durante o trabalho foram notadas brechas em que tecnologias alinhadas a este modelo poderiam trazer ainda mais eficácia. Porém como o objetivo do trabalho já havia sido alcançado, não houve necessidade de aprofundamento nestas melhoras.

Assim encorajamos aqueles que darão continuidade a este trabalho que busquem melhorias como a integração com sistemas de monitoramento constante (Prometheus, Grafana), a automação de coleta de dados, treinamento e previsão do modelo através de pipelines CI/CD, realização de testes com outros conjuntos de dados e a modificação das camadas do algoritmo LSTM, utilizando de camadas de regularização como Batch Normalization, para estabilizar o treinamento e acelerar a convergência do modelo, ou a ampliação das camadas densas após as camadas LSTM para realizar transformações lineares nos dados e aprender representações mais complexas.

5 - Bibliografia

[1] Huang, J., Chen, G. and Cheng, B. (2016), A Stochastic Approach of Dependency Evaluation for IoT Devices. Chinese J. Electron., 25: 209-214. https://doi.org/10.1049/cje.2016.03.003

[2] ARANTES, R. C.; PEREIRA, M. M. O.; CASTRO, C. C., MINEIRO, A. A. C.; OLIVEIRA, J. A. A transformação digital e o conhecimento organizacional: Uma revisão sistemática da literatura. Contextus – Revista Contemporânea de Economia e Gestão, Fortaleza, v. 19, n.21, p. 316- 329, jan./dez., 2021.
https://doi.org/10.19094/contextus.2021.71301

[3] Coulouris, George, Jean Dollimore, and Tim Kindberg. Sistemas distribuídos. Vol. 6. Madrid: Addison Wesley, 2001.

[4] ROTHSCHILD, Michael. Colonial Pipeline Ransomware Attack: How to Reduce Risk in OT Environments. Tenable, 14 maio 2021. Disponível em: https://pt-br.tenable.com/blog/colonial-pipeline-ransomware-attack-how-to-reduce-risk-in-ot-environments. Acesso em: 09 de Maio de 2024.

[5] Virpathi, Anuj & Pathak, Heman. (2018). A Comparative Study of Grid Load Balancing. International Journal of Computer Applications. 179. 25-33. 10.5120/ijca2018916315.
[6] Pahl, Claus & Jamshidi, Pooyan. (2016). Microservices: A Systematic Mapping Study. 137-146. 10.5220/0005785501370146.

[7] de Azevedo, Leonildo & Estrella, Júlio & Nakamura, Luis H. V. & Santana, Marcos & Santana, Richard & Toledo, Claudio & Batista, Bruno & Reiff-Marganiec, Stephan. (2018). Optimized Service Level Agreement Establishment in Cloud Computing. Computer Journal. 61. 1429-1442. 10.1093/comjnl/bxx087.

[8] Wazlawick, R. S. (2017). Metodologia de Pesquisa para Ciência da Computação. Rio de Janeiro: Elsevier Brasil.

[9] Oliveira, T. P., Barbar, J. S., & Soares, A. S. (2015). Predição do tráfego de rede de computadores usando redes neurais tradicionais e de aprendizagem profunda. Revista De Informática Teórica E Aplicada, 22(1), 10–28. https://doi.org/10.22456/2175-2745.49045

[10] MACEDO, Evandro Luiz Cardoso. Previsão de tráfego em enlaces de redes utilizando séries temporais. Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Engenharia de Sistemas e Computação, COPPE, da Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Mestre em Engenharia de Sistemas e Computação. Rio de Janeiro, 2015. 98 p. Disponível em: https://www.cos.ufrj.br/uploadfile/publicacao/Dissertacao_Evandro_Final_v201510021.pdf.

[11] SILVA, Henrique Dornel da; CHRISTO, Eliane da Silva. Previsão de Carga em Redes de Computadores Utilizando Séries Temporais. In: SBPO - Simpósio Brasileiro de Pesquisa Operacional, 2016, Florianópolis. Anais do SBPO. Florianópolis: SBPO, 2016. p. 1-10. Disponível em: https://www.din.uem.br/sbpo/sbpo2016/pdf/162652.pdf.

[12] AMARAL, Matheus Basso do. Previsão de Velocidades de Carga com Rede Neural LSTM. Dissertação (Mestrado em Informática) - Universidade Federal do Rio Grande do Sul, Porto Alegre, 2020. Disponível em: https://lume.ufrgs.br/bitstream/handle/10183/218100/001121611.pdf?sequence=1&isAllowed=y.

[13] CORTEZ, Paulo; RIO, Miguel; ROCHA, Miguel; SOUSA, Pedro. Multi-scale Internet Traffic Forecasting using Neural Networks and Time Series Methods. IEEE Transactions on Network and Service Management, v. 7, n. 3, p. 480-494, 2010. DOI: 10.1111/j.1468-0394.2010.00568.x.

[14] COMUNIDADE CLOUD. Auto-scaling. Disponível em: https://comunidadecloud.com/auto-scaling/. Acesso em: 15 mai. 2024.

[15] JOSHI, Manish R.; HADI, Theyazn Hassn. A Review of Network Traffic Analysis and Prediction Techniques. School of Computer Sciences, North Maharashtra University, Jalgaon (M.S) India. Disponível em: https://www.researchgate.net/publication/339927785_A_review_of_network_traffic_analysis_and_prediction_techniques. Acesso em: 15 mai. 2024.

[16] IBM. Auto-scaling. Disponível em: https://www.ibm.com/topics/autoscaling. Acesso em: 15 mai. 2024.

[17] AMAZON WEB SERVICES. Auto Scaling. Disponível em: https://aws.amazon.com/pt/autoscaling/. Acesso em: 15 mai. 2024.

[18] TELES, Guilherme. Escala automática (Auto-scaling). Medium, 21 mai. 2019. Disponível em: https://medium.com/@telesguilherme/escala-autom%C3%A1tica-auto-scaling-8739be4b86c3. Acesso em: 15 mai. 2024.

[19] AMAZON WEB SERVICES. Documentação do Amazon Auto Scaling - Visão geral das políticas de dimensionamento em etapas. Disponível em: https://docs.aws.amazon.com/pt_br/autoscaling/application/userguide/step-scaling-policy-overview.html. Acesso em: 15 mai. 2024.

[20] AMAZON WEB SERVICES. Documentação do Amazon EC2 Auto Scaling - Dimensionamento simples em etapas. Disponível em: https://docs.aws.amazon.com/pt_br/autoscaling/ec2/userguide/as-scaling-simple-step.html

[21] AWARI. Algoritmo de Aprendizado de Máquina: O Que É. Disponível em: https://awari.com.br/algoritmo-de-aprendizado-de-maquina-o-que-e/. Acesso em: 2 jun. 2024.

[22] EXPRESS ANALYTICS. Neural Networks Prediction. Disponível em: https://www.expressanalytics.com/blog/neural-networks-prediction/. Acesso em: 2 jun. 2024.

[23] WANG Shihao, ZHUO Qinzheng, YAN Han, LI Qianmu, QI Yong. A Network Traffic Prediction Method Based on LSTM. [J], 2019, 17(2): 19-25 doi:10.12142/ZTECOM.201902004

[24] AZZOUNI, Abdelhadi; PUJOLLE, Guy. A Long Short-Term Memory Recurrent Neural Network Framework for Network Traffic Matrix Prediction. Paris: LIP6 / UPMC. Disponível em: https://arxiv.org/pdf/1705.05690. Acesso em: 2 jun. 2024.

[25] AMAZON WEB SERVICES. O que é uma rede neural?. Disponível em: https://aws.amazon.com/pt/what-is/neural-network/. Acesso em: 2 jun. 2024.

[26] XAVIER, Miguel G.; MATTEUSSI, Kassiano J.; LORENZO, Fabian; DE ROSE, Cesar A. F. Understanding performance interference in multi-tenant cloud databases and web applications. Pontifical Catholic University of Rio Grande do Sul (PUCRS), Faculty of Informatics, Porto Alegre, Brazil. Disponível em: https://meriva.pucrs.br/dspace/bitstream/10923/23208/2/Understanding_performance_interference_in_multitenant_cloud_databases_and_web_applications.pdf. Acesso em: 31 mai. 2024.

[27] GRAFANA. Load Testing Websites. Disponível em: https://grafana.com/blog/2024/01/30/load-testing-websites/. Acesso em: 31 mai. 2024.

[28] Križanić, J., Grguric, A., Mosmondor, M., & Lazarevski, P. (2010). Load testing and performance monitoring tools in use with AJAX based web applications. The 33rd International Convention MIPRO, 428-434.

[29] Chen, D., Song, Q., Zhang, Y., Li, L., & Yang, Z. (2023). Journal of Sensors, 2023, 1–9. doi:10.1155/2023/5997304 https://onlinelibrary.wiley.com/doi/10.1155/2023/5997304

[30] Scikit-learn. Decision Trees. Disponível em: https://scikit-learn.org/stable/modules/tree.html. Acesso em: 13 jun. 2024.

[31] SOUZA, E. P. D. O. (2024). Análise comparativa de modelos de machine learning e convolutional neural networks na detecção de falhas em máquinas rotativas (Master's thesis, Universidade Federal de Pernambuco). https://repositorio.ufpe.br/handle/123456789/55791

[32] BROWNLEE, J. ARIMA for time series forecasting with Python. Disponível em: https://machinelearningmastery.com/arima-for-time-series-forecasting-with-python/. Acesso em: 13 jun. 2024.

[33] Facebook Prophet. Quick Start. Disponível em: https://facebook.github.io/prophet/docs/quick_start.html. Acesso em: 13 jun. 2024.

[34] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30. https://arxiv.org/abs/1706.03762

[35] MWITI, D. Google 2019 cluster sample. Disponível em: https://www.kaggle.com/datasets/derrickmwiti/google-2019-cluster-sample. Acesso em: 13 jun. 2024.

[36] L. Z. N. Costa, L. A. Rodovalho, L. B. A. P. dos Santos, e P. H. M. da Silva, "Desenvolvimento de Modelos e Gráficos em Sistemas Distribuidos", Google Colab, 2024. Disponível em: https://colab.research.google.com/drive/1tCzw3pFyGk03EcJt8zvI5fzR7jJyljDc?usp=sharing. Acesso em: 17 jun. 2024

[37] Medium. MAE, MSE, RMSE, Coefficient of Determination, Adjusted R Squared. Disponível em: https://medium.com/analytics-vidhya/mae-mse-rmse-coefficient-of-determination-adjusted-r-squared-which-metric-is-better-cd0326a5697e. Acesso em: 13 jun. 2024.

[38] Medium. Prevendo números: entendendo métricas de regressão. Disponível em: https://medium.com/data-hackers/prevendo-n%C3%BAmeros-entendendo-m%C3%A9tricas-de-regress%C3%A3o-35545e011e70. Acesso em: 13 jun. 2024.

[39] Elmachtoub, A. N., Liang, J. C. N., McNellis, R. Decision Trees for Decision-Making under the Predict-then-Optimize Framework. arXiv:2003.00360 [cs.LG]. Disponível em: https://doi.org/10.48550/arXiv.2003.00360.

[40] Saha, A., Basu, S., Datta, A. Random Forests for dependent data. arXiv:2007.15421 [stat.ML]. Disponível em: https://doi.org/10.48550/arXiv.2007.15421.

[41] Li, Z., Liu, L., Dong, C., Shang, J. Overfitting or Underfitting? Understand Robustness Drop in Adversarial Training. arXiv:2010.08034 [cs.LG]. Disponível em: https://doi.org/10.48550/arXiv.2010.08034.

Acompanhamento do Professor

Data: 05/05/2024

Menção de acompanhamento: MM

O trabalho ainda carece de citações e referências para respaldar o conteúdo. A ideia está boa, mas carece de fundamentação .

Menção Final de Partida : MS

- Não foi demonstrada de forma prática a questão da estabilidade conforme apontado no título e objetivo. Sequer foi conceituado o que é estabilidade de sistemas.
- Pouca discussão sobre desafios de implementação: O texto não aborda em detalhes os desafios práticos de implementar e manter um sistema de IA para previsão de carga em um ambiente de produção real, como a necessidade de coleta contínua de dados, ajuste do modelo e gerenciamento de recursos
- Na conclusão ainda há a discussão de relatos de teste e observações que deveriam estar no capítulo de desenvolvimento.
-